

Análise Artigo

Introdução ao Problema da Modularização

O artigo "On the Criteria To Be Used in Decomposing Systems into Modules", escrito por D.L. Parnas, é um texto fundamental na engenharia de software. O autor foca na modularização como um mecanismo essencial para melhorar a flexibilidade e a compreensibilidade de um sistema, além de permitir a redução do tempo de desenvolvimento.

Parnas argumenta que a eficácia da modularização depende estritamente dos critérios utilizados para dividir o sistema em módulos. Para ilustrar sua tese, ele apresenta um problema de design de sistema e contrasta uma decomposição convencional com uma decomposição não convencional.

O Sistema de Exemplo: Índice KWIC

Para demonstrar as diferenças entre os critérios de modularização, o autor utiliza um sistema de produção de índice KWIC (Keyword In Context).

- O sistema KWIC aceita um conjunto ordenado de linhas, onde cada linha é um conjunto de palavras e cada palavra é um conjunto de caracteres.
- O sistema "desloca circularmente" as linhas, removendo a primeira palavra e anexando-a ao final da linha repetidamente.
- O resultado final é uma listagem de todos os deslocamentos circulares de todas as linhas em ordem alfabética.
- Embora seja um sistema pequeno que poderia ser programado em uma ou duas semanas, Parnas o trata como um grande projeto para ilustrar as dificuldades que motivam a programação modular.

As Duas Abordagens de Modularização

Parnas apresenta duas formas distintas de dividir este mesmo sistema. Em ambas as abordagens, a definição de "módulo" é tratada como uma atribuição de responsabilidade, e não apenas como um subprograma.

Modularização 1: A Abordagem Convencional (Baseada em Fluxograma)

Esta é a decomposição que a maioria dos programadores proporia, baseada nas etapas de processamento. É análoga a criar um fluxograma do sistema.

- **Módulo 1 (Input):** Lê os dados, armazena-os na memória principal, empacota caracteres quatro por palavra e mantém um índice do endereço inicial de cada linha.
- **Módulo 2 (Circular Shift):** Prepara um índice com o endereço do primeiro caractere de cada deslocamento circular e o índice original da linha, gerando pares de dados na memória.
- **Módulo 3 (Alphabetizing):** Recebe os arrays dos módulos 1 e 2 e produz um novo array com os deslocamentos circulares em ordem alfabética.
- **Módulo 4 (Output):** Usa os arrays produzidos para gerar uma listagem bem formatada.
- **Módulo 5 (Master Control):** Controla o sequenciamento entre os outros módulos.

Modularização 2: A Abordagem Não Convencional (Ocultação de Informação)

Nesta abordagem, os módulos não correspondem a etapas de processamento. Cada módulo é caracterizado pelo conhecimento de uma decisão de design que ele esconde dos outros módulos.

- **Módulo 1 (Line Storage):** Fornece funções para armazenar e acessar caracteres e palavras (ex: \$CHAR(r,w,c)\$, \$SETCHAR(r,w,c,d)\$), escondendo completamente como esses dados são fisicamente armazenados na memória.
- **Módulo 2 (INPUT):** Lê as linhas originais e chama o módulo de armazenamento para guardá-las.
- **Módulo 3 (Circular Shifter):** Fornece funções análogas ao Módulo 1 (ex: \$CSCHAR(l,w,c)\$), criando a ilusão de que todos os deslocamentos circulares estão armazenados, sem revelar como são calculados ou mantidos.
- **Módulo 4 (Alphabetizer):** Consiste em funções que permitem acessar o enésimo deslocamento na ordem alfabética, escondendo o momento e o método exato da alfabetização.
- **Módulo 5 (Output) e Módulo 6 (Master Control):** Têm funções de impressão e controle de fluxo semelhantes à primeira modularização.

Comparação e Vantagens da Ocultação de Informação

Embora ambos os sistemas possam funcionar e até compartilhar os mesmos algoritmos de base, eles diferem drasticamente em sua capacidade de adaptação e manutenção.

Critério de Avaliação	Modularização 1 (Convencional)	Modularização 2 (Ocultação de Informação)
Mudança no armazenamento (ex: usar disco em vez de memória)	Exigiria mudanças em todos os módulos, pois todos dependem do formato exato de armazenamento.	A mudança fica confinada inteiramente ao módulo <i>Line Storage</i> .
Mudança no empacotamento de caracteres	Afetaria múltiplos módulos que acessam os arrays.	Totalmente invisível para o resto do sistema, restrito ao <i>Line Storage</i> .
Momento da Alfabetização (ex: busca sob demanda)	Difícil de implementar, pois o módulo de saída espera que o índice alfabético esteja pronto antes de começar.	Nenhuma alteração fora do módulo <i>Alphabetizer</i> seria necessária.
Desenvolvimento Independente	Requer que o formato das tabelas e estruturas de dados seja rigorosamente definido em conjunto antes que o trabalho independente comece.	As interfaces são abstratas (nomes de funções e parâmetros), permitindo que o desenvolvimento independente comece muito mais cedo.
Compreensibilidade	O sistema só faz sentido como um todo; entender a saída exige entender o armazenamento e a alfabetização.	Cada módulo pode ser estudado e compreendido de forma independente.